

LAB GUIDE WITH SOLUTIONS

Redis as a Cache

Commands, configuration parameters and patterns - with solutions

Module 07 · NoSQL & Redis

ICICI Bank Learning Academy

Enterprise Data Engineering with SQL, Redis & Kafka

Module 07 · NoSQL & Redis · Instructor-led · For classroom use

How to use this guide

This guide teaches Redis **as a cache** - not as a database - using nothing but `redis-cli`. Sections 1 and 2 give you the vocabulary and every configuration parameter that turns Redis into a cache. Labs C.1 to C.7 then walk through the commands one at a time, each with a **Try it** task and its solution. Section 10 is the pattern catalogue, and section 11 shows the same commands being issued by the Horizon Bank demo application (companion: `Horizon_Bank_App_Guide.pdf`).

Part	Title	Time
1	Caching in one page - hit, miss, TTL, eviction, hit ratio	10 min
2	Cache configuration parameters - <code>redis.conf</code> and <code>CONFIG SET</code>	20 min
C.1	Keys with a lifetime - <code>SET EX</code> , <code>TTL</code> , <code>EXPIRE</code> , <code>PERSIST</code> , <code>KEEPTTL</code> , <code>GETEX</code>	25 min
C.2	Cache-aside by hand - <code>GET</code> , miss, <code>SET EX</code> , hit	15 min
C.3	Hash caches and per-field expiry - <code>HSET</code> , <code>HEXPIRE</code> , <code>HTTL</code>	15 min
C.4	Invalidation - <code>DEL</code> , <code>UNLINK</code> , <code>SCAN MATCH</code>	15 min
C.5	How expiry really happens - lazy and active expiry	15 min
C.6	Eviction under memory pressure - noeviction vs LRU vs volatile-LRU vs LFU	30 min
C.7	Measuring the cache - <code>INFO</code> stats, hit ratio, <code>MONITOR</code> , <code>SLOWLOG</code>	20 min
10	Caching patterns and pitfalls	20 min
11	The same commands inside the Horizon Bank app	15 min
12	Cheat sheet	-

SETUP Start a throw-away Redis for the labs. It is deliberately separate from the app's bank-redis container so that Lab C.6 (which fills memory on purpose) cannot disturb the application.

```
$ docker run -d --name redis-lab redis:7-alpine
$ docker exec -it redis-lab redis-cli
127.0.0.1:6379> INFO server
  redis_version:7.4.11

# when you are finished with all labs
$ docker rm -f redis-lab
```

READ THIS Conventions: lines starting with `$` run in your shell (Git Bash, WSL or PowerShell). Lines starting with `127.0.0.1:6379>` run inside `redis-cli`. Teal lines are the output you should see - captured from a real run, not typed by hand. TTL values will differ by a second or two on your machine.

SECTION 1

Caching in one page

A **cache** is a small, fast copy of data whose real home is somewhere slower. In the Horizon Bank app the real home is PostgreSQL - it is durable, transactional and the only place a balance is ever *changed*. Redis holds a copy of the answers the dashboard asks for most often, in RAM, so the next identical question is answered without touching the database.

Term	Meaning	Redis command / metric
Cache hit	The answer was already in Redis	GET returns a value; keyspace_hits +1
Cache miss	Not in Redis - go to the database, then store it	GET returns (nil); keyspace_misses +1
TTL	Time-to-live: seconds until Redis deletes the key by itself	SET k v EX 300, TTL k, EXPIRE k 300
Expiry	Removal because the TTL ran out	expired_keys in INFO stats
Eviction	Removal because memory is full (maxmemory reached)	maxmemory-policy; evicted_keys
Invalidation	Removal by the application because the data changed	DEL / UNLINK
Hit ratio	hits / (hits + misses) - the one number that says if a cache works	INFO stats
Stale data	A cached copy that no longer matches the database	bounded by TTL + invalidation

Why Redis is a good cache

- **In-memory** - reads and writes are measured in microseconds inside the server; the network round trip usually dominates.
- **Expiry is built in** - every key can carry its own TTL; Redis deletes it for you.
- **Memory is bounded** - maxmemory plus an eviction policy means a full cache throws away cold keys instead of crashing.
- **Rich values** - a cached item can be a string (JSON), a hash (field-level reads), a list, a sorted set - not just a blob.
- **Shared** - every API instance sees the same cache, unlike an in-process Map.

What should - and should not - be cached

Good candidates	Poor candidates
Read far more often than written (statements, account lists)	Written on almost every read
Expensive to compute (joins, aggregates)	Cheap primary-key lookups on a small table
Tolerates a few seconds of staleness	Must be exact at every instant (the debit itself)
Same answer for many requests	Unique per request, never asked twice

IMPORTANT The balance that decides whether a transfer is allowed is **always** read from PostgreSQL with a row lock - never from the cache. The cache serves the *display* of balances and statements. Hold on to that rule; section 10 comes back to it.

SECTION 2

Cache configuration parameters

Two settings turn Redis from a database into a cache: a memory ceiling and a policy for what to throw away when that ceiling is reached. Everything else is tuning. The table lists every directive in `cache_banking_app/redis/redis-cache.conf`, the file the app's Redis container boots with.

Directive	App value	Default	What it controls
<code>maxmemory</code>	64mb	0 (no limit)	Ceiling for the dataset. 0 means grow until the OS kills the process
<code>maxmemory-policy</code>	<code>allkeys-lru</code>	<code>noeviction</code>	What to evict when maxmemory is hit - see Lab C.6
<code>maxmemory-samples</code>	5	5	Keys sampled per eviction; LRU/LFU are approximations. 10 = closer to exact
<code>lazyfree-lazy-eviction</code>	yes	no	Free evicted values in a background thread
<code>lazyfree-lazy-expire</code>	yes	no	Free expired values in a background thread
<code>lazyfree-lazy-user-del</code>	yes	no	Make DEL behave like UNLINK (non-blocking)
<code>hz</code>	10	10	Background task frequency - drives active expiry (Lab C.5)
<code>save</code>	""	<code>3600 1 300 100 60 10000</code>	RDB snapshots. Off - a cache can be rebuilt from the DB
<code>appendonly</code>	no	no	AOF log. Off for the same reason
<code>timeout</code>	0	0	Close idle clients after N s. 0 = never (the API holds one connection)
<code>tcp-keepalive</code>	300	300	Detect dead peers
<code>slowlog-log-slower-than</code>	10000	10000	Log commands slower than 10 ms (microseconds)
<code>latency-monitor-threshold</code>	100	0	Record latency spikes over 100 ms
<code>lfu-log-factor</code> / <code>lfu-decay-time</code>	(default)	10 / 1	Only for *-lfu policies: counter growth and decay

WATCH OUT `redis.conf` does not allow a comment on the same line as a directive. `maxmemory 64mb # cap` makes Redis refuse to start with "wrong number of arguments". Put comments on their own line.

Reading and changing configuration at runtime

```
127.0.0.1:6379> CONFIG GET maxmemory
 1) "maxmemory"
 2) "0"
127.0.0.1:6379> CONFIG GET maxmemory-policy
 1) "maxmemory-policy"
 2) "noeviction"
127.0.0.1:6379> CONFIG GET maxmemory-samples
 1) "maxmemory-samples"
 2) "5"
127.0.0.1:6379> CONFIG SET maxmemory 2mb
OK
```

```
127.0.0.1:6379> CONFIG SET maxmemory-policy allkeys-lru
OK

# the app's container, booted from redis-cache.conf
$ docker exec bank-redis redis-cli CONFIG GET maxmemory-policy
maxmemory-policy
allkeys-lru

$ docker exec bank-redis redis-cli CONFIG GET maxmemory
maxmemory
67108864
```

A fresh `redis:7-alpine` container reports `maxmemory 0` and `noeviction` - it behaves like a database: it never throws data away, and once the host runs out of RAM it fails. `CONFIG SET` changes apply instantly but are lost on restart unless you also run `CONFIG REWRITE` (which needs a writable config file) or put them in `redis.conf` - which is what the app does.

Application-level cache parameters

The API adds its own parameters on top - they decide *what* gets cached and for how long. They are environment variables in `docker-compose.yml`:

Variable	Default	Meaning
CACHE_ENABLED	true	false = every request goes to PostgreSQL (compare the timings)
CACHE_TTL_SECONDS	300	Base TTL of every cached statement / account list
CACHE_TTL_JITTER_SECONDS	30	Random 0..30 s added to each TTL so keys do not expire together
CACHE_KEY_PREFIX	bank	Namespace for every key: bank:txns:..., bank:accounts:...
CACHE_ON_WRITE	refresh	After a transfer: refresh (write-through) or invalidate (delete)
STATEMENT_SIZE	50	How many recent transactions one cached statement holds

LAB C.1 · 25 MINUTES

Keys with a lifetime

Objective	Prerequisite
Create keys that expire, read their remaining life, extend it, remove it, and update a value without resetting its TTL.	redis-lab running; redis-cli open

Step 1 - set a value with a 60-second TTL and inspect it. EX is seconds, PX is milliseconds.

```
127.0.0.1:6379> SET bank:bal:602501000101 111053.04 EX 60
OK
127.0.0.1:6379> GET bank:bal:602501000101
"111053.04"
127.0.0.1:6379> TTL bank:bal:602501000101
(integer) 59
127.0.0.1:6379> PTTL bank:bal:602501000101
(integer) 58698
```

Step 2 - change the lifetime. EXPIRE replaces the TTL; PERSIST removes it.

```
127.0.0.1:6379> EXPIRE bank:bal:602501000101 300
(integer) 1
127.0.0.1:6379> TTL bank:bal:602501000101
(integer) 300
127.0.0.1:6379> PERSIST bank:bal:602501000101
(integer) 1
127.0.0.1:6379> TTL bank:bal:602501000101
(integer) -1
127.0.0.1:6379> TTL bank:nosuchkey
(integer) -2
```

TTL returns	Meaning
n > 0	Key exists and expires in n seconds
-1	Key exists and has NO expiry - in a cache this is usually a bug
-2	Key does not exist (never set, expired, evicted or deleted)

Step 3 - the overwrite trap and KEeptTL. A plain SET on an existing key *removes* its TTL. KEeptTL keeps it.

```
127.0.0.1:6379> EXPIRE bank:bal:602501000101 120
(integer) 1
127.0.0.1:6379> SET bank:bal:602501000101 108453.04 KEeptTL
OK
127.0.0.1:6379> TTL bank:bal:602501000101
(integer) 119
```

Step 4 - conditional writes and GETEX. NX = only if missing, XX = only if present. GETEX reads and re-arms the TTL in one command (sliding expiry).

```
127.0.0.1:6379> SET bank:bal:602501000101 999 NX
(nil)      <- key exists, not written
127.0.0.1:6379> SET bank:bal:602501000999 999 XX
(nil)      <- key missing, not written
127.0.0.1:6379> GETEX bank:bal:602501000101 EX 600
"108453.04"
```

```
127.0.0.1:6379> TTL bank:bal:602501000101
(integer) 600
127.0.0.1:6379> SETEX bank:otp:9820011223 30 482913
OK
127.0.0.1:6379> EXISTS bank:bal:602501000101 bank:nosuchkey
(integer) 1
127.0.0.1:6379> TYPE bank:bal:602501000101
string
127.0.0.1:6379> OBJECT ENCODING bank:bal:602501000101
"embstr"
```

TRY IT Cache a customer's display name under `bank:name:1` for 45 seconds, but only if it is not already cached. Then update the value to "Arjun K. Mehta" without losing the remaining TTL.

Solution

```
127.0.0.1:6379> SET bank:name:1 "Arjun Mehta" EX 45 NX
OK
127.0.0.1:6379> SET bank:name:1 "Arjun K. Mehta" KEPTTL XX
OK
127.0.0.1:6379> TTL bank:name:1
(integer) 44
```

Running the first command a second time returns (nil): NX refuses because the key is already there.

CHECKPOINT You can explain what TTL -1 and -2 mean, and why a plain SET on a cached key is dangerous.

LAB C.2 · 15 MINUTES

Cache-aside by hand

Objective	Prerequisite
Play the application's part: look in the cache, miss, load from the database, store with a TTL, hit.	Lab C.1

Cache-aside (also called *lazy loading*) is the pattern the Horizon Bank API uses for every read. The application - not Redis - is responsible for filling the cache.

```
# 1. the app asks the cache first
127.0.0.1:6379> GET bank:txns:602501000101
(nil)                                <- MISS
# 2. ...runs SELECT ... FROM transactions in PostgreSQL, then stores the JSON
127.0.0.1:6379> SET bank:txns:602501000101
'["txnId":91,"type":"DEBIT","amount":203.78,"description":"UPI/BookMyShow"]' EX 300
OK
# 3. the next identical request
127.0.0.1:6379> GET bank:txns:602501000101
'["txnId":91,"type":"DEBIT","amount":203.78,...]' <- HIT
127.0.0.1:6379> TTL bank:txns:602501000101
(integer) 299
```

Step	Who does it	Command
Look up	Application	GET key
On miss: load	Application	SQL against PostgreSQL
On miss: fill	Application	SET key value EX ttl
Expire	Redis, automatically	(TTL reaches 0)
Invalidate on change	Application	DEL key - or SET again (write-through)

TRY IT Why does the SET in step 2 carry EX 300? What would happen over a month if it did not?

Solution

```
# Without a TTL the key lives until it is evicted or deleted.
# Any change the app forgot to invalidate would be served forever,
# and every account ever viewed would stay in RAM. TTL is the safety net
# that bounds staleness to at most 300 s even when invalidation fails.
```

CHECKPOINT You can name the three things the application does on a miss, and the one thing Redis does on its own.

LAB C.3 · 15 MINUTES

Hash caches and per-field expiry

Objective	Prerequisite
Cache an object as a hash so single fields can be read, and expire a single field (Redis 7.4+).	Lab C.1

A JSON string is read and written as one blob. A **hash** lets the app read just balance without transferring the whole object. The TTL is still set on the key.

```
127.0.0.1:6379> HSET bank:acct:602501000101 holder "Arjun Mehta" type SAVINGS balance 111053.04
branch "Andheri West"
(integer) 4
127.0.0.1:6379> EXPIRE bank:acct:602501000101 300
(integer) 1
127.0.0.1:6379> HGET bank:acct:602501000101 balance
"111053.04"
127.0.0.1:6379> HGETALL bank:acct:602501000101
1) "holder"
2) "Arjun Mehta"
3) "type"
4) "SAVINGS"
5) "balance"
6) "111053.04"
7) "branch"
8) "Andheri West"
```

Since Redis 7.4, a single field can have its own TTL - useful when one field (the balance) goes stale faster than the rest (the holder's name):

```
127.0.0.1:6379> HEXPIRE bank:acct:602501000101 60 FIELDS 1 balance
1) (integer) 1
127.0.0.1:6379> HTTL bank:acct:602501000101 FIELDS 2 balance holder
1) (integer) 60
2) (integer) -1
```

Store as	Pros	Cons
String (JSON)	One GET/SET; easy to serialise from any language	Must rewrite the whole value to change one field
Hash	Read/update single fields; per-field TTL (7.4+)	Nested data must be flattened

The Horizon Bank app stores statements as JSON strings: a statement is always read whole, so a hash would add work without saving any.

TRY IT Add a field lastTxn = "TRF/To 602501000201" to the hash and make only that field expire in 30 seconds.

Solution

```
127.0.0.1:6379> HSET bank:acct:602501000101 lastTxn "TRF/To 602501000201"
(integer) 1
127.0.0.1:6379> HEXPIRE bank:acct:602501000101 30 FIELDS 1 lastTxn
1) (integer) 1
```

CHECKPOINT You can choose between a string and a hash for a cached object and justify it.

LAB C.4 · 15 MINUTES

Invalidation - removing what changed

Objective	Prerequisite
Delete a single cached key, then find and delete a family of keys safely.	Lab C.2

```

127.0.0.1:6379> DEL bank:txns:602501000101
(integer) 1
127.0.0.1:6379> GET bank:txns:602501000101
(nil)          <- next read will be a MISS and reload
127.0.0.1:6379> SET bank:txns:602501000102 x EX 300
OK
127.0.0.1:6379> SET bank:txns:602501000103 x EX 300
OK
127.0.0.1:6379> SCAN 0 MATCH bank:txns:* COUNT 100
1) "0"
2) 1) "bank:txns:602501000102"
   2) "bank:txns:602501000103"
127.0.0.1:6379> UNLINK bank:txns:602501000102 bank:txns:602501000103
(integer) 2

```

Command	Use it for	Note
DEL k [k...]	Removing known keys	Frees memory on the main thread - blocks on huge values
UNLINK k [k...]	Removing large keys	Returns at once; memory freed in a background thread
SCAN cursor MATCH p	Finding keys by pattern	Incremental; repeat until the cursor is "0"
KEYS pattern	Never in production	Walks the whole keyspace in one blocking call
FLUSHALL	Never on a shared server	Deletes every key in every database

DESIGN The best invalidation never needs a pattern search: design keys so the code that changes the data can *compute* every key it affects. A transfer from A to B knows it must touch `bank:txns:A`, `bank:txns:B` and the two owners' `bank:accounts:*` keys - no SCAN needed.

TRY IT From the shell (not redis-cli), delete every `bank:txns:*` key in one pipeline without using KEYS.

Solution

```

$ docker exec redis-lab sh -c "redis-cli --scan --pattern 'bank:txns:*' | xargs -r redis-cli UNLINK"
3          <- keys removed; no "(integer)" prefix when redis-cli is not on a terminal

```

CHECKPOINT You can explain why KEYS is banned in production and what replaces it.

LAB C.5 · 15 MINUTES

How expiry really happens

Objective	Prerequisite
Watch a key expire, and understand the two mechanisms Redis uses to delete expired keys.	Lab C.1

```
127.0.0.1:6379> SET bank:short 1 EX 2
OK
127.0.0.1:6379> GET bank:short
"1"
# ... wait 3 seconds ...
127.0.0.1:6379> GET bank:short
(nil)
127.0.0.1:6379> INFO keyspace
# Keyspace
db0:keys=3,expires=3,avg_ttl=235911,subexpiry=1
```

Mechanism	When it runs	What it does
Lazy (passive) expiry	Whenever a client touches a key	If the key is past its TTL, delete it and answer as if it never existed
Active expiry	hz times per second (default 10)	Sample keys that have a TTL, delete the expired ones, repeat while >10% were expired

Together they guarantee an expired key is never *returned*, and that memory is reclaimed even for keys nobody reads again. In `INFO keyspace`, `expires=3` counts keys that carry a TTL; in a healthy cache it should be equal (or close) to `keys`. `subexpiry=1` is the hash with a per-field TTL from Lab C.3.

TRY IT How many expired keys has this server removed so far?

Solution

```
$ docker exec redis-lab redis-cli INFO stats | grep expired_keys
expired_keys:1
```

CHECKPOINT You can explain why a TTL of 300 s does not mean the memory is freed at exactly 300 s.

LAB C.6 · 30 MINUTES

Eviction under memory pressure

Objective	Prerequisite
Fill a 2 MB Redis with 5,000 x 1 KB keys under three policies and compare what survives.	Labs C.1-C.5

Step 1 - noeviction (the default). Cap memory at 2 MB and push 5,000 SET bank:txns:<n> <1000 bytes> EX 300 commands through redis-cli --pipe, exactly as a busy application would.

```
127.0.0.1:6379> CONFIG SET maxmemory 2mb
OK
127.0.0.1:6379> CONFIG SET maxmemory-policy noeviction
OK
$ docker exec -i redis-lab sh -c 'i=1; while [ $i -le 5000 ]; do \
    printf "SET bank:txns:%d %01000d EX 300\r\n" $i 0; i=$((i+1)); done | redis-cli --pipe'
errors: 4089, replies: 5000
127.0.0.1:6379> DBSIZE
(integer) 911
127.0.0.1:6379> SET bank:one-more hello
(error) OOM command not allowed when used memory > 'maxmemory'.
```

Once full, **every write fails** - 4,089 of the 5,000. For a cache this is the worst outcome: the application's SET after a miss errors, so the hottest new data can never get in.

Step 2 - allkeys-lru. FLUSHALL, set the policy, run the same 5,000 SETs.

```
127.0.0.1:6379> CONFIG SET maxmemory-policy allkeys-lru
OK
$ (same 5000-SET pipeline)
errors: 0, replies: 5000
127.0.0.1:6379> DBSIZE
(integer) 875
127.0.0.1:6379> GET bank:txns:1
(nil) <- oldest, evicted
127.0.0.1:6379> EXISTS bank:txns:5000
(integer) 1 <- newest, kept
$ docker exec redis-lab redis-cli INFO stats | grep evicted_keys
evicted_keys:4192
```

Step 3 - volatile-lru and a key with no TTL. Only keys that carry an expiry are eviction candidates.

```
127.0.0.1:6379> CONFIG SET maxmemory-policy volatile-lru
OK
127.0.0.1:6379> SET bank:config:limits '{"maxTransfer":200000}'
OK <- no EX: not evictable
$ (same 5000-SET pipeline)
errors: 0, replies: 5000
127.0.0.1:6379> EXISTS bank:config:limits
(integer) 1
127.0.0.1:6379> DBSIZE
(integer) 854
```

Policy	Evicts from	Chooses by	Use when
noeviction	nothing - writes fail	-	Redis is a database (queues, sessions you cannot lose)
allkeys-lru	all keys	least recently used	Pure cache - the Horizon Bank setting
allkeys-lfu	all keys	least frequently used	Cache with a stable set of hot keys
allkeys-random	all keys	random	Uniform access, lowest CPU
volatile-lru	keys with a TTL	least recently used	Cache + permanent keys in one instance
volatile-lfu	keys with a TTL	least frequently used	Same, frequency-based
volatile-ttl	keys with a TTL	shortest TTL left	Items that are about to expire anyway
volatile-random	keys with a TTL	random	Rarely the right choice

WATCH OUT With a volatile- * policy and no keys carrying a TTL, Redis has nothing it is allowed to evict and behaves exactly like noeviction.

Step 4 - see what LRU and LFU look at.

```
127.0.0.1:6379> CONFIG SET maxmemory-policy allkeys-lfu
OK
127.0.0.1:6379> SET bank:txns:hot x
OK
127.0.0.1:6379> SET bank:txns:cold x
OK
# GET bank:txns:hot executed 50 times
127.0.0.1:6379> OBJECT FREQ bank:txns:hot
(integer) 10
127.0.0.1:6379> OBJECT FREQ bank:txns:cold
(integer) 5
127.0.0.1:6379> CONFIG SET maxmemory-policy allkeys-lru
OK
127.0.0.1:6379> SET bank:txns:idle x
OK
# ... 4 seconds later
127.0.0.1:6379> OBJECT IDLETIME bank:txns:idle
(integer) 4
127.0.0.1:6379> CONFIG SET maxmemory 0
OK <- undo the lab limit
```

The LFU counter is logarithmic (50 reads -> 10, not 50) and starts at 5 so new keys are not evicted instantly. OBJECT FREQ only works under an LFU policy; OBJECT IDLETIME (seconds since last access) is what LRU uses.

TRY IT A team runs sessions (must never be lost, no TTL) and a statement cache (TTL 300 s) in the same Redis. Which policy, and why?

Solution

```
CONFIG SET maxmemory-policy volatile-lru
# Only TTL-carrying cache keys can be evicted; sessions without a TTL are safe.
# Better still: run two Redis instances, each with its own policy.
```

CHECKPOINT You have seen noeviction refuse writes, allkeys-lru keep the newest data, and volatile-lru protect a key that has no TTL.

LAB C.7 · 20 MINUTES

Measuring the cache

Objective	Prerequisite
Compute the hit ratio, count expirations and evictions, and watch live traffic.	Labs C.1-C.6

```
$ docker exec redis-lab redis-cli INFO stats | grep -E 'keyspace_|expired_keys|evicted_keys'
expired_keys:1
evicted_keys:0
keyspace_hits:17
keyspace_misses:5

# hit ratio = hits / (hits + misses) = 17 / 22 = 77.3 %
$ docker exec redis-lab redis-cli CONFIG RESETSTAT      # zero the counters
OK
```

Metric	Healthy	Investigate when
hit ratio	> 80 % for a read-heavy cache	Falls after a deploy (key names changed?) or TTL cut
evicted_keys	0 or slowly rising	Climbs fast: maxmemory too small for the working set
expired_keys	Rising steadily	Flat while keys pile up: TTLs are missing (TTL -1)
used_memory vs maxmemory	Below the limit	Pinned at the limit with evictions = undersized
slowlog	Empty	KEYS, big DEL, huge values

Watching live traffic

MONITOR prints every command the server receives. It is the best teaching tool there is and a terrible production tool - it can halve throughput. Stop it with Ctrl+C.

```
$ docker exec -it bank-redis redis-cli MONITOR
[0 172.20.0.4:36098] "GET" "bank:txns:602501000101"
[0 172.20.0.4:36098] "INCR" "bank:stats:misses"
[0 172.20.0.4:36098] "SET" "bank:txns:602501000101" "[{...}]" "EX" "329"
[0 172.20.0.4:36098] "GET" "bank:txns:602501000101"
[0 172.20.0.4:36098] "TTL" "bank:txns:602501000101"
[0 172.20.0.4:36098] "INCR" "bank:stats:hits"

$ docker exec bank-redis redis-cli SLOWLOG GET 5
(empty array)
```

TRY IT Your cache reports keyspace_hits:900 and keyspace_misses:600, with evicted_keys climbing by thousands a minute. What is the hit ratio, and what is the first thing to change?

Solution

```
# 900 / 1500 = 60 %.
# Rapid eviction means the working set does not fit: raise maxmemory
# (or cache less per key) before touching the TTL.
CONFIG SET maxmemory 256mb
```

CHECKPOINT You can read a Redis server's health as a cache from four numbers in INFO.

SECTION 10

Caching patterns and pitfalls

Pattern	Read path	Write path	Trade-off
Cache-aside (lazy)	App: GET; on miss load DB + SET	App writes DB, then DEL or SET key	Simple, resilient; first read after a change is slow
Read-through	Cache library loads from DB itself	-	Cleaner app code; needs a smart cache layer
Write-through	Always a hit after a write	Write DB, then SET the new value	Fresh reads; writes do more work - Horizon Bank's default
Write-behind	From cache	Write cache; DB updated later, async	Fast writes; data loss risk - never for money
Refresh-ahead	From cache	Background job re-loads keys before TTL ends	No miss latency; loads data nobody may ask for

Pitfall	Symptom	Fix	Redis command
Stampede (dog-pile)	Hot key expires; 500 requests all miss and hit the DB together	One loader holds a short lock; others wait or serve stale	SET lock:k id NX EX 5
Avalanche	Many keys written together expire together	Add random jitter to TTLs	SET k v EX 300+rand(30)
Penetration	Requests for IDs that do not exist always miss	Cache the "not found" for a short TTL	SET k "null" EX 30
Stale after write	UI shows the old balance	Update/delete keys after COMMIT	SET / DEL
Update before COMMIT	Cache shows a transfer that rolled back	Touch the cache only after COMMIT	-
Missing TTL	Memory grows forever; TTL k = -1	Every cache SET carries EX	SET k v EX n

The stampede lock, by hand

```
127.0.0.1:6379> SET lock:bank:txns:602501000101 api-1 NX EX 5
OK          <- api-1 won; it loads from the DB
127.0.0.1:6379> SET lock:bank:txns:602501000101 api-2 NX EX 5
(nil)       <- api-2 lost; wait briefly, then GET again
```

IMPORTANT Money moves only in PostgreSQL. The transfer reads the balance with SELECT ... FOR UPDATE, not from Redis, so a stale cache can at worst *display* an old figure for a few seconds - it can never let an account go overdrawn.

SECTION 11

The same commands inside the Horizon Bank app

Start the application (see `Horizon_Bank_App_Guide.pdf`), open MONITOR on its Redis in one terminal, and run the demo script in another:

```
$ cd cache_banking_app && docker compose up -d --build
$ docker exec -it bank-redis redis-cli MONITOR          # terminal 1
$ bash scripts/cache-demo.sh                          # terminal 2

== 2. First enquiry -> MISS, loaded from PostgreSQL, stored in Redis
source=database ttl=300s took=22.2ms
== 3. Same enquiry again -> HIT, served from Redis
source=cache ttl=300s took=2.4ms
== 4. What Redis now holds
bank:stats:hits
bank:txns:602501000101
bank:stats:misses
TTL bank:txns:602501000101 = 299
TYPE = string
== 5. Transfer Rs 100 to 602501000201 -> DB COMMIT, then cache refreshed
{"reference":"TRF295949631951",...,"cacheUpdates":[
  {"key":"bank:txns:602501000101","action":"refreshed","ttl":305},
  {"key":"bank:txns:602501000201","action":"refreshed","ttl":302},
  {"key":"bank:accounts:1","action":"refreshed","ttl":304},
  {"key":"bank:accounts:2","action":"refreshed","ttl":327}]}
== 6. Enquire again -> still a HIT, and the newest row is the transfer
"source":"cache"
"description":"TRF/To 602501000201 - cli demo"
```

Key	Holds	Set by	Updated when
bank:txns:<accountNo>	Latest 50 transactions (JSON)	GET /accounts/:no/transactions on a miss	A transfer touches the account
bank:accounts:<customerId>	Accounts + balances (JSON)	GET /customers/:id/accounts on a miss	A transfer touches any of the customer's accounts
bank:stats:hits / misses	Counters (INCR)	Every cached read	Never expire - reset by Clear cache

Measured on this machine, 100 requests each against the savings account's statement: a **MISS** (PostgreSQL + SET) took **3.3 ms** at the median, a **HIT 0.9 ms** - with only ~500 rows in a warm, idle database. The gap widens as the database gets bigger and busier; the cache's real job is to keep that load off PostgreSQL at all.

TRY IT Switch the app to invalidate-on-write and show that the first statement read after a transfer is a MISS.

Solution

```
$ CACHE_ON_WRITE=invalidate docker compose up -d api
$ curl -s -X POST localhost:4000/api/transfers -H 'Content-Type: application/json' \
  -d '{"fromAccount":"602501000101","toAccount":"602501000103","amount":50}' \
  ... "cacheUpdates":[{"key":"bank:txns:602501000101","action":"deleted","ttl":null},...]
$ curl -s localhost:4000/api/accounts/602501000101/transactions | cut -c1-60
{"source":"database","key":"bank:txns:602501000101","ttl":30
$ docker compose up -d api          # back to the default: refresh
```

SECTION 12

Cheat sheet

Task	Command
Cache a value for 5 minutes	<code>SET k v EX 300</code>
...only if not cached yet	<code>SET k v EX 300 NX</code>
Update, keep remaining TTL	<code>SET k v KEPTTL</code>
Read and extend (sliding TTL)	<code>GETEX k EX 300</code>
Seconds / ms left	<code>TTL k / PTTL k</code>
Change / remove TTL	<code>EXPIRE k 300 / PERSIST k</code>
Per-field TTL on a hash (7.4+)	<code>HEXPIRE k 60 FIELDS 1 f / HTTL k FIELDS 1 f</code>
Invalidate	<code>DEL k / UNLINK k</code>
Find keys safely	<code>SCAN 0 MATCH bank:* COUNT 100</code>
Memory ceiling	<code>CONFIG SET maxmemory 64mb</code>
Eviction policy	<code>CONFIG SET maxmemory-policy allkeys-lru</code>
Hit / miss counters	<code>INFO stats (keyspace_hits, keyspace_misses)</code>
Evictions / expirations	<code>INFO stats (evicted_keys, expired_keys)</code>
Keys with TTL	<code>INFO keyspace (keys=, expires=)</code>
Memory used by one key	<code>MEMORY USAGE k</code>
LRU idle / LFU counter	<code>OBJECT IDLETIME k / OBJECT FREQ k</code>
Watch live commands (dev only)	<code>MONITOR</code>
Slow commands	<code>SLOWLOG GET 10</code>
Stampede lock	<code>SET lock:k id NX EX 5</code>
Reset counters	<code>CONFIG RESETSTAT</code>

NEXT Next: open 09_Redis_Caching.pptx for the classroom version of this guide, and Horizon_Bank_App_Guide.pdf to see how the app is built around these commands.